Демидовский университет

Модель-вид-делегат

Автор: доцент, кандидат технических наук
Ксения Владимировна Лагутина

Архитектура модель-вид

- Модель-вид — архитектура представления данных в графическом интерфейсе
  - Общий класс для вида и контроллера
- Виджеты таблиц, списков и деревьев могут получить доступ к данным двумя способами:
  - Виджеты включают в себя внутренние контейнеры для хранения данных
    - Подход приводит к проблемам с синхронизацией данных
  - Программирование модель-вид, виджеты получают доступ к внешним данным через стандартизированный интерфейс

https://doc.qt.io/qtforpython-6/overviews/qtwidgets-modelview.html

https://doc.qt.io/qtforpython-6/overviews/qtwidgets-model-view-programming.html

Демидовский университет

Виджеты для простых списков

- QListWidget – список
  - addItem(label), addItems(labels) – добавить элементы
  - itemAt() – получить элемент по его позиции
  - itemWidget(item) – получить виджет элемента
- QTableWidget – таблица
  - setItem(row, column, item) – добавить элемент в ячейку
  - setCellWidget(row, column, widget) – добавить виджет в ячейку
- QTreeWidget – дерево

https://doc.qt.io/qtforpython/PySide6/QtWidgets/QListWidget.html

https://doc.qt.io/qtforpython/PySide6/QtWidgets/QTableWidget.html

Демидовский университет

Пример простого списка

```
from PySide6.QtWidgets import QApplication, QAbstractItemView,
QListWidget

app = QApplication([])
numbers = ["One", "Two", "Three", "Four", "Five"]
listWidget = QListWidget()
listWidget.setSelectionMode(QAbstractItemView.SingleSelection)
listWidget.addItems(numbers)
listWidget.show()
app.exec()
```

Демидовский университет

Архитектура модель-вид-делегат

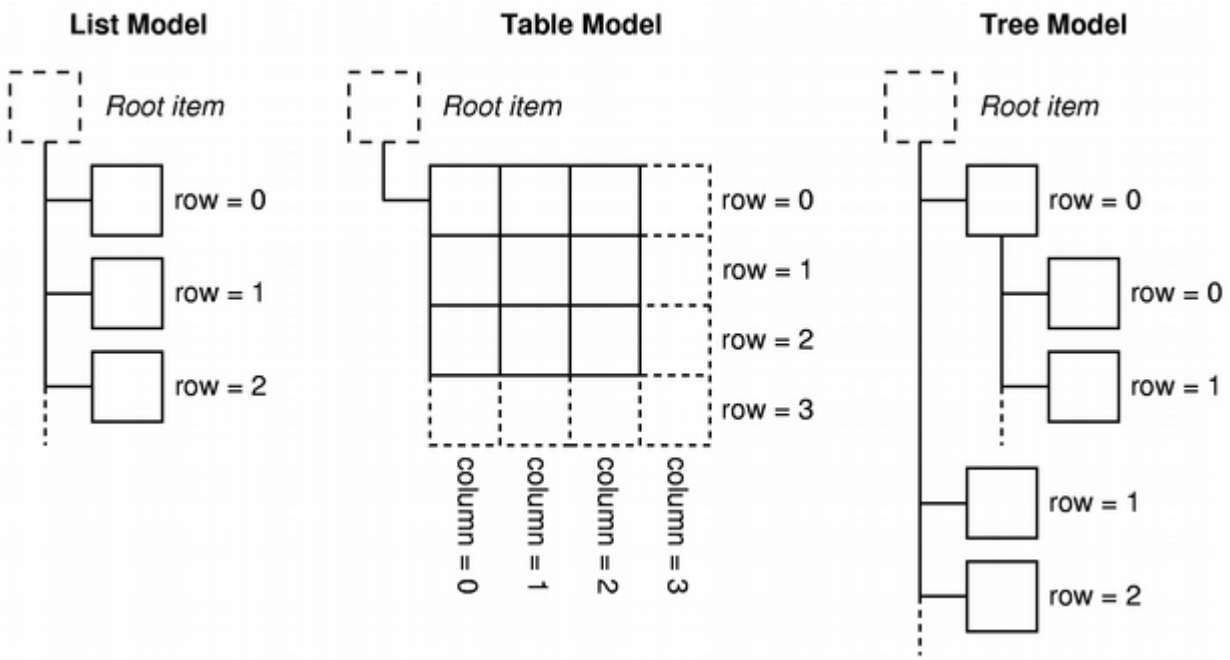
- Модель взаимодействует с источником данных
- Вид получает индексы из модели; это ссылки на элементы данных
- Делегат позволяет настраивать способ отображения и редактирования элементов данных
- Когда элемент редактируется, делегат связывается с моделью напрямую, используя индексы модели
- Сигналы от модели информируют вид об изменениях в данных
- Сигналы от вида предоставляют информацию о взаимодействии пользователя с отображаемыми элементами
- Сигналы от делегата используются во время редактирования, чтобы сообщить модели и виду о состоянии редактора

Модели

- QAbstractItemModel — интерфейс для работы с видами, представляющими данные в виде таблиц, списков и деревьев
  - Базовый класс моделей
- QAbstractListModel и QAbstractTableModel — списки и таблицы
- Готовые модели:
  - QStringListModel хранит простой список элементов QString
  - QStandardItemModel управляет древовидными структурами элементов, каждый из которых может содержать произвольные данные
  - QFileSystemModel предоставляет информацию о файлах и каталогах
  - QSqlQueryModel, QSqlTableModel и QSqlRelationalTableModel используются для доступа к базам данных

Индексы моделей

- Индексы предоставляют временные ссылки на элементы модели и могут использоваться для извлечения или изменения данных через модель
- Индексы отделяют модель от вида
  - Только модель должна знать, как получить данные
- Индексы модели содержат указатель на модель, которая их создала
- К элементам модели можно обращаться по номерам строк и столбцов
  - Реальный способ хранения данных может отличаться
- index = model.index(row, column, parent)
- indexA = model.index(0, 0, QModelIndex())



Демидовский университет

Роли

- Роль различает разные виды данных, связанных с элементом
- Возможные роли:
  - DisplayRole — отображение данных
  - EditRole — изменение данных
  - DecorationRole — фон ячейки в виде иконки
  - ToolTipRole, StatusTipRole, WhatsThisRole — подсказки

Демидовский университет

QAbstractItemModel — базовый класс

- index() — индекс модели
- data()/setData() — возвращает/устанавливает данные модели по индексу и роли
- flags() — способы манипуляций элементами
- rowCount() и columnCount() — количество строк и столбцов
- insertRows(), insertColumns(), removeRows() и removeColumns() — управление строками и столбцами
- headerData()/setHeaderData() — заголовки
- dataChanged() испускается, когда данные изменяются

Демидовский университет

https://doc.qt.io/qtforpython/PySide6/QtCore/QAbstractItemModel.html

QStringListModel — строки

- setStringList(strings) — установить список строк
- stringList() — получить список строк
- Нет оповещения об изменениях

https://doc.qt.io/qtforpython/PySide6/QtCore/QStringListModel.html

Демидовский университет

QStandardItemModel — стандартные типы

- appendRow() — добавить элемент в модель
- item() — доступ к элементу
- setRowCount() и setColumnCount() — изменение размеров модели
- Нет оповещения об изменениях

https://doc.qt.io/qtforpython/PySide6/QtGui/QStandardItemModel.html

Демидовский университет

Виды

- QListView – список
- QTableView – таблица
- QTreeView – дерево элементов
- QColumnView показывает дерево как иерархию списков
- QComboBox может быть и классическим виджетом, и видом
- QAbstractItemView – базовый класс
  - setModel(model) – указать модель
  - clicked(index) – сигнал нажатия

https://doc.qt.io/qtforpython/PySide6/QtWidgets/QAbstractItemView.html

Пример QListView

```
class Window(QMainWindow):
    def __init__(self):
        super(Window, self).__init__()
        listView = QListView()
        self.data = ['Item 1', 'Item 2', 'Item 3', 'Item 4']
        listView.setModel(QStringListModel(self.data))
        listView.clicked.connect(self.clicked)
        self.setCentralWidget(listView)

    def clicked(self, modelIndex):
        print('You have selected:' + self.data[modelIndex.row()])

if __name__ == '__main__':
    app = QApplication([])
    window = Window()
    window.show()
    app.exec()
```

Демидовский университет

Пример QTableView

```
class MyModel(QAbstractTableModel):
    def __init__(self, parent=None):
        super().__init__(parent)

    def rowCount(self, parent=None):
        return 2

    def columnCount(self, parent=None):
        return 3

    def data(self, index, role=Qt.DisplayRole):
        if role == Qt.DisplayRole:
            row = index.row() + 1
            column = index.column() + 1
            return f"Row{row}, Column{column}"
        return None
if __name__ == '__main__':
    app = QApplication([])
    table_view = QTableView()
    my_model = MyModel()
    table_view.setModel(my_model)
    table_view.show()
    app.exec()
```

Демидовский университет

Пример с классом поезда

- Пример выложен в Moodle
- Демонстрирует, как вывести объекты своего класса в список и таблицу
- Демонстрирует удаление из списка
- Демонстрирует настойку ширины колонок и блокирование редактора

Демидовский университет

QSortFilterProxyModel – фильтрация и сортировка

- Прокси-класс преобразует структуру исходной модели, сопоставляя исходные индексы с новыми, соответствующими результатам фильтрации или сортировки
- Прокси-класс позволяет реструктурировать исходную модель, не требуя никаких преобразований базовых данных и не дублируя данные
- setSourceModel() — задать исходную модель
- sort() — сортировка по столбцу в прямом или обратном порядке
  - lessThan() — можно переопределить, чтобы задать способ сравнения
- setFilterKeyColumn() — колонка, но основе которой идёт фильтрация
- setFilterFixedString(query) — фильтрация по наличию подстроки query
- setFilterRegularExpression() — фильтрация по регулярному выражению
- filterAcceptsRow() — задать способ фильтрации
  - invalidateFilter() — обновить фильтрацию. Вызывается, когда собственные параметры фильтра изменились

Демидовский университет

https://doc.qt.io/qtforpython-6/PySide6/QtCore/QSortFilterProxyModel.html

Делегаты

- Предоставляют возможности ввода
- Отвечают за отображение отдельных элементов в видах
- QAbstractItemDelegate – базовый класс
- QStyledItemDelegate – простой визуальный делегат

QAbstractItemDelegate — делегат

- paint() – событие рисования, отображает делегат
- Два подхода к редактированию
  - Переопределить createEditor() – создать виджет редактора, setEditorData() –заполнить редактор данными из модели, setModelData() – обновлять модель данными из редактора
  - Обработка пользовательских событий напрямую путем переопределения editorEvent()

https://doc.qt.io/qtforpython/PySide6/QtWidgets/QAbstractItemDelegate.html

Демидовский университет

Пример собственного делегата

```
class SpinBoxDelegate(QStyledItemDelegate):
    def __init__(self, parent=None):
        super().__init__(parent)
    def createEditor(self, parent, option, index):
        editor = QSpinBox(parent)
        editor.setFrame(False)
        editor.setMinimum(0)
        editor.setMaximum(100)
        return editor
    def setEditorData(self, editor, index):
        value = index.model().data(index,
Qt.EditRole)
        editor.setValue(value)
    def setModelData(self, editor, model, index):
        editor.interpretText()
        value = editor.value()
        model.setData(index, value, Qt.EditRole)
    def updateEditorGeometry(self, editor, option,
index):
        editor.setGeometry(option.rect)
```

```
if __name__ == '__main__':
    app = QApplication([])
    model = QStandardItemModel(4, 2)
    tableView = QTableView()
    tableView.setModel(model)
    delegate = SpinBoxDelegate()
    tableView.setItemDelegate(delegate)
    tableView.horizontalHeader().setStretchLastSection(True)
    for row in range(4):
        for column in range(2):
            index = model.index(row, column, QModelIndex())
            value = (row + 1) * (column + 1)
            model.setData(index, value)
    tableView.setWindowTitle("Spin Box Delegate")
    tableView.show()
    app.exec()
```

Демидовский университет